
CASE STUDY / 2026

RevupAds

Ad-revenue analytics dashboard for digital publishers.

React 19 • PHP • MySQL — Multi-tenant analytics & reporting platform

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	System Architecture	Page 4
03	Data Flow — Upload to Chart	Page 5
04	Report Types & API Endpoints	Page 6
05	Technology Stack	Page 7
06	The Challenge	Page 8
07	The Approach & Role Matrix	Page 9
08	Code Spotlight	Page 10
09	Outcome & Engagement	Page 11

This document presents an in-depth look at the RevupAds platform — what it does, how data moves through it, the role-based separation between admin and client, and the engineering decisions behind it. Every diagram, code snippet, and technical claim is grounded directly in the project source.

SECTION 01

Project Overview

What RevupAds is, who it serves, and the shape of the platform.

RevupAds is a multi-tenant ad-revenue analytics dashboard built for a digital-ads agency and the publishers it manages. The agency-side **Admin** uploads spreadsheet exports from each ad network (display, video, payments) into a central database. Each **Client** — a publisher tied to a specific website — logs in and sees their own data, narrowed to their site, presented as searchable tables and dual-axis charts.

The platform is a single React 19 SPA backed by a PHP REST API on the same host, with MySQL as the system of record. Authentication uses cookie-backed PHP sessions, and access is gated at the route level by a *ProtectedRoute* component that consults *checkSession.php* on mount. The admin uploads CSV or XLSX files, which are parsed entirely in the browser using papaparse / SheetJS before being persisted row-by-row through the relevant *insert_** endpoint.

The system covers three distinct report types — **Display Ads**, **Video Ads**, and **Payments** — each with its own schema, its own upload flow, its own admin table view, and (for the first two) its own dual-axis Recharts visualisation on the client side.

3

Report types

7

PHP endpoints

2

User roles

Recharts

Dual-axis charts

SECTION 02

System Architecture

Single SPA, one PHP API, one MySQL database, two role-gated views.

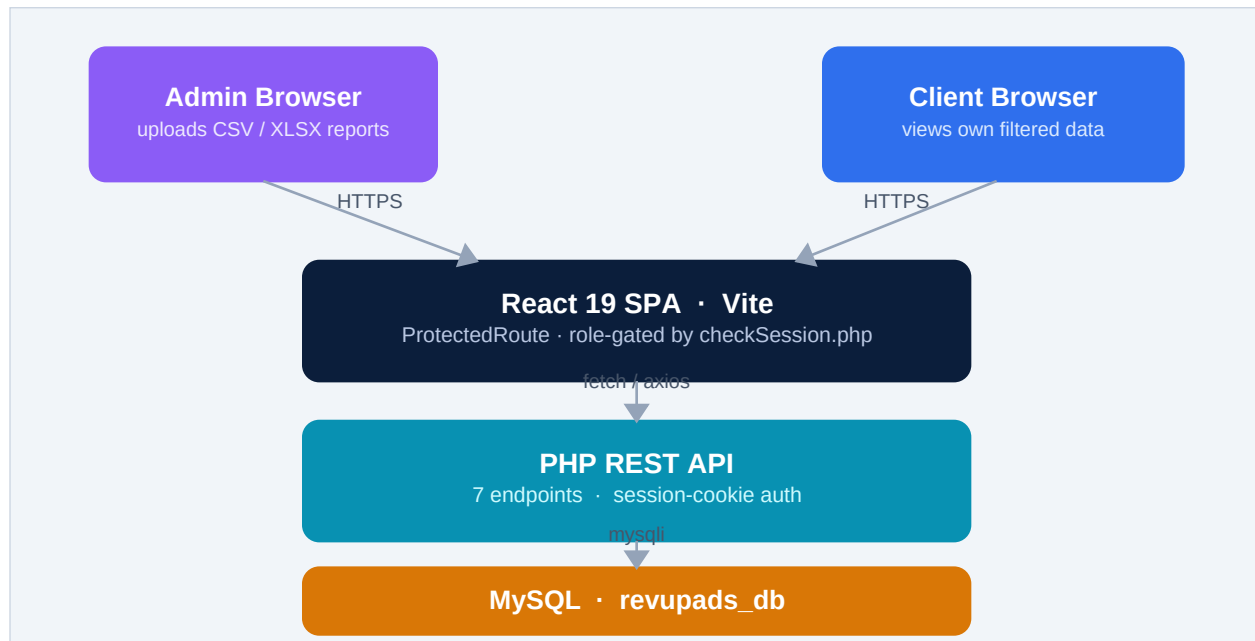


Figure — Admin uploads through the SPA; clients view through the SPA; both hit the same PHP API, both share the same MySQL store.

Architecturally the system is deliberately flat. The React 19 SPA is the single client; on first paint it calls *checkSession.php* to determine whether the user is an admin, a client, or unauthenticated. *ProtectedRoute* hard-routes them to */admin*, */client*, or back to login accordingly.

The PHP REST API is a thin layer over *mysql* with prepared statements; every endpoint is single-purpose. Session cookies carry identity; *credentials: 'include'* on the frontend keeps them attached to every fetch. CORS is locked to *https://www.revupads.com* on production endpoints. MySQL holds three report tables, a clients table, and any auxiliary lookup data.

SECTION 03

Data Flow — Upload to Chart

Two parallel paths: the admin loads data in, the client pulls data out.

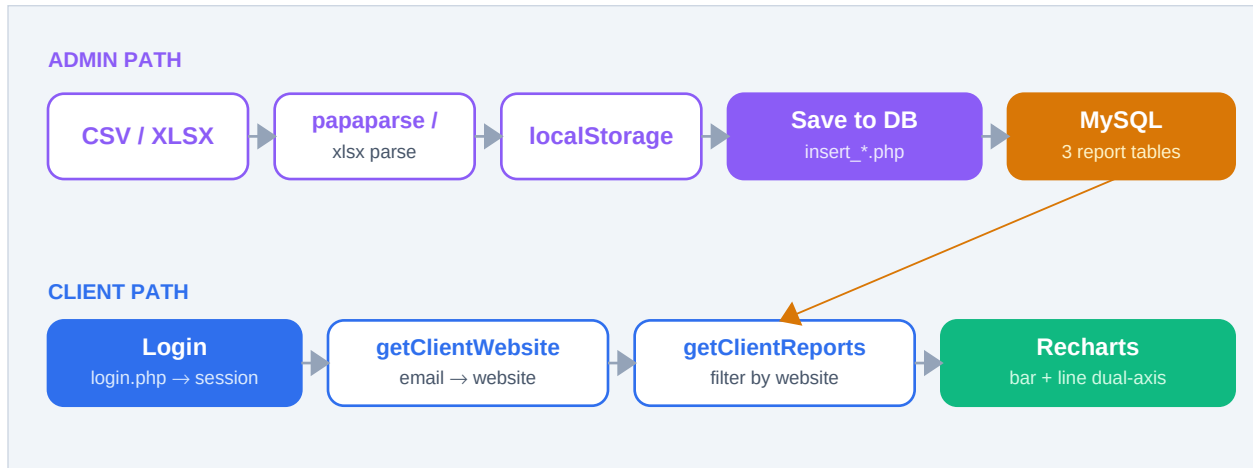


Figure — Admin: CSV/XLSX → in-browser parse → localStorage → save-to-DB → MySQL. Client: login → website lookup → filtered reports → Recharts.

Admin path. The admin picks a CSV or XLSX. The frontend detects the file type by extension and routes to either *papaparse* (CSV with header inference) or *SheetJS / xlsx* for Excel — including a custom helper that converts Excel's serial numeric dates back into ISO strings. The parsed rows are validated against a required-fields whitelist, written into *localStorage* as a preview, and only persisted to the database when the admin clicks *Save to Database*. Each row goes through the matching *insert_*.php* endpoint individually, with error reporting on the first failure.

Client path. The client logs in via *login.php*, which sets the session cookie and returns the role plus their website. The frontend then calls *getClientWebsite.php?email=<email>* to confirm, and *getClientReports.php?website=<website>* to pull every report row scoped to that publisher. The response feeds a *Recharts* dual-axis chart — impressions as bars, earnings as a line — plus a table view filtered by date or month.

SECTION 04

Report Types & API Endpoints

Three distinct schemas, seven typed endpoints.

Report types

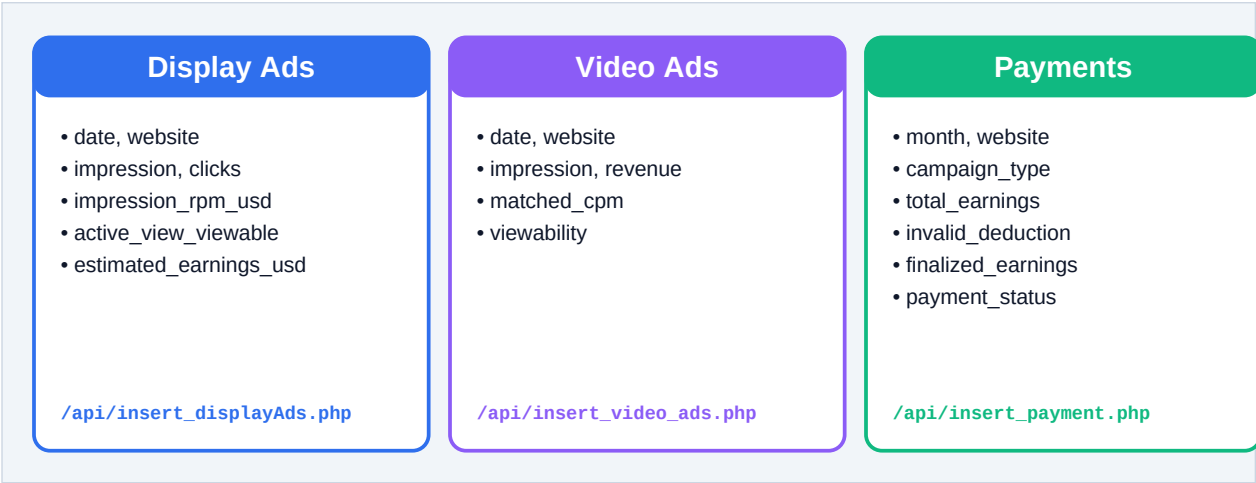


Figure — Three report types, each with its own schema and its own insert endpoint.

REST endpoints

Method	Endpoint	Purpose
POST	<code>/api/login.php</code>	Validate credentials, assign role, start session
GET	<code>/api/checkSession.php</code>	Probe — returns loggedIn flag + email/website
GET	<code>/api/getClientWebsite.php</code>	Look up the website tied to a client's email
GET	<code>/api/getClientReports.php</code>	Return all reports filtered by website
POST	<code>/api/insert_displayAds.php</code>	Persist parsed display-ad rows
POST	<code>/api/insert_video_ads.php</code>	Persist parsed video-ad rows
POST	<code>/api/insert_payment.php</code>	Persist parsed payment rows

SECTION 05

Technology Stack

Modern React on the frontend, classic LAMP-style PHP on the backend.

Layer	Technology & Details
Frontend	React 19, Vite 6, react-router-dom 7, react-hot-toast, axios
Charts	Recharts 2 — dual-axis bar + line for Display Ads and Video Ads
File parsing	<i>papaparse</i> for CSV, <i>SheetJS (xlsx)</i> for Excel — with Excel serial-date conversion
Auth helper	Firebase / firebase-admin v11 (for optional client identity), session cookies for primary auth
Backend	PHP — single-purpose REST scripts under <i>/api/</i> , <i>mysqli</i> + prepared statements
Database	MySQL — <i>revupads_db</i> in production (<i>revupads_local</i> for dev)
Edge	Hosted at https://www.revupads.com ; CORS allow-list locked to the production domain
Persistence	localStorage preview before commit; uploaded data survives accidental tab close

Why this stack. React 19 and Vite 6 give a fast modern SPA with strict TypeScript-grade ESLint rules; recharts handles dual-axis visualisations without a third-party charting service. PHP + MySQL keeps the backend within shared-hosting reach — no Node runtime, no containers — while still giving prepared-statement safety and CORS controls. Session cookies (with HttpOnly and SameSite=Lax) keep auth simple and resilient.

SECTION 06

The Challenge

An agency manages many publishers; each publisher only ever sees their own data.

An ad-revenue agency reconciling Google Ad Manager, video ad networks, and monthly payment exports for many publishers ends up with a folder full of CSVs and spreadsheets that the publishers never see. Every "how did my site do last month?" question becomes a manual lookup, filter, and screenshot. At any scale beyond a handful of clients, that operational tax overwhelms the actual revenue work.

The brief was a platform that lets the agency upload reports once and gives each client a self-serve view of *only their own site's* performance. Five constraints made it non-trivial: **(1)** reports arrive in two file formats (CSV and XLSX) and three different schemas; **(2)** Excel dates land as serial numbers, not strings; **(3)** clients must be strictly prevented from seeing other clients' data; **(4)** the agency wants chart visualisations, not just tables; and **(5)** upload mistakes should be recoverable — a wrong file shouldn't poison the database.

Operationally the platform also had to run on standard PHP / MySQL hosting — no Node service, no microservices — so deployment is a git pull on the host plus a database migration.

SECTION 07

The Approach & Role Matrix

Browser-side parsing, server-side persistence, hard-gated routing, per-website scoping.

Role-access matrix

	Admin	Client
Upload Display-Ads CSV/XLSX	✓	—
Upload Video-Ads CSV/XLSX	✓	—
Upload Payments CSV/XLSX	✓	—
View all reports	✓	—
View own website's reports	—	✓
Date / month filter	✓	✓
Charts (Recharts)	—	✓ dual-axis
Session-protected route	/admin	/client

Figure — Admin uploads everything; clients see only their own filtered data.

The system uses three layers of separation. **Routing** is gated by *ProtectedRoute*, which redirects unauthenticated users to login and unauthorised users away from the wrong dashboard. **API** is gated by session cookies — every endpoint checks the session before answering. **Data** is gated at the query level: *getClientReports.php* only ever returns rows whose website matches the session-bound website. A client could craft a request with a different website parameter and the SQL would still scope to their own.

Upload safety comes from a two-step flow: parse first, persist second. Files are parsed in the browser using papaparse (for CSV) or SheetJS (for XLSX); a required-fields whitelist rejects malformed uploads before they leave the browser; a localStorage preview lets the admin spot-check before committing. Only when the admin clicks *Save to Database* does the row-by-row insert run.

SECTION 08

Code Spotlight

Two production snippets — the PHP login flow, and the browser-side Excel-date conversion.

PHP login — prepared statement, role assignment, session bootstrap

api/login.php

```
// login.php — credential check + session + role assignment
$stmt = $conn->prepare("SELECT * FROM clients WHERE email = ? AND password = ?");
$stmt->bind_param("ss", $email, $password);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows === 1) {
    $user = $result->fetch_assoc();
    $role = ($user['email'] === 'mudasirshafiq07@gmail.com')
        ? 'admin' : 'client';
    $_SESSION['email'] = $user['email'];
    $_SESSION['role'] = $role;
    echo json_encode([
        "status" => "success",
        "role" => $role,
        "website" => $user['website'],
    ]);
}
```

CSV / XLSX upload — Excel-date handling + dual-format parsing

src/components/DisplayAdsUpload.jsx

```
// Excel serial-date → ISO date (handles xlsx uploads cleanly)
function excelDateToJSDate(serial) {
    const utc_days = Math.floor(serial - 25569);
    const d = new Date(utc_days * 86400 * 1000);
    return `${d.getUTCFullYear()}-${('0' + (d.getUTCMonth()+1)).slice(-2)}-${('0' + d.getUTCDate()).slice(-2)}`;
}

// Parse CSV via papaparse, XLSX via SheetJS, then validate required fields
if (extension === 'csv') {
    Papa.parse(file, { header: true, skipEmptyLines: true,
        complete: (r) => parseData(r.data) });
} else if (extension === 'xlsx') {
    const workbook = XLSX.read(data, { type: 'array' });
    const rows = XLSX.utils.sheet_to_json(workbook.Sheets[
        workbook.SheetNames[0]], { defval: '' });
    parseData(rows);
}
```

SECTION 09

Outcome & Engagement

What the platform delivers, and how to take the conversation further.

RevupAds turns a folder of weekly ad-network CSVs into a self-serve client experience. Publishers can answer their own "how did my site do?" questions on demand; the agency stops being the bottleneck for routine reporting. Three distinct report types live behind one consistent upload flow and one consistent client view.

Operationally the platform is built to be extended without rewriting. Adding a fourth report type means a new `insert_*.php`, a new upload component, and an entry on the admin Tabs control — every other layer (auth, session, ProtectedRoute, getClientReports, Recharts rendering) is reused as-is.

- › **Three report types** — Display Ads, Video Ads, Payments — each with its own schema and dedicated upload flow.
- › **Browser-side parsing** — papaparse for CSV, SheetJS for XLSX, with custom Excel serial-date conversion.
- › **Two-step upload safety** — preview in localStorage, then commit row-by-row only on explicit Save to Database.
- › **Session-cookie auth** — HttpOnly, SameSite=Lax cookies; ProtectedRoute enforces role-based routing on the SPA.
- › **Per-website scoping** — getClientReports.php filters by the session-bound website so clients only ever see their own data.
- › **Dual-axis Recharts** — bars for impressions, line for earnings; per-date filter dropdown for narrowing.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks React • PHP / MySQL • Spring Boot • Python • BigQuery • Docker